



CHALMERS
UNIVERSITY OF TECHNOLOGY

CA 1 - Incompressible Elasticity

TME250 - Finite Element Method: Solids

Academic year 2025/2026, SP2

Sankalp Manojkumar Prabhu, sankalpm@chalmers.se

November 21, 2025

Contents

1	Task 1	1
1.1	Problem Description	1
1.2	Computing relation between T and U	1
1.2.1	Case I: $\epsilon_{11} = 0$	1
1.2.2	Case II: $\sigma_{11} = 0$	2
2	Task 2	3
2.1	Problem Description	3
2.2	Methodology	3
2.3	Result and Analysis	4
2.3.1	Effect of Poisson's ratio	5
2.3.2	Mesh Convergence Study	6
2.3.3	Pressure Distribution	6
3	Task 3	9
3.1	Problem Description	9
3.2	Methodology	9
3.3	Result and Analysis	10
3.3.1	Uniform Traction Load Case	10
3.3.2	Prescribed Displacement Load Case	10
3.3.3	Mesh Convergence Study	11
3.3.4	Pressure Distribution	11
4	Conclusion	14
A	Appendix: Task 2 Code List	I
A.1	Main Code	I
A.2	Load Interpolation For Linear Elements	XI
B	Appendix: Task 3 Code List	XII
B.1	Main Code	XII
B.2	Load Interpolation for Quadratic Elements	XXII
B.3	Taylor Hood Element Routine	XXIII
B.3.1	Stiffness Matrix	XXIII
B.3.2	Supporting Code	XXIII

1

Task 1

1.1 Problem Description

A two-dimensional domain is considered which modelled under the plain strain condition and subjected to symmetric boundary condition on the left and bottom edges.

For computing the approximate relation between T and U as a function of ν , two extreme cases of $\epsilon_{11} = 0$ and $\sigma_{11} = 0$ together with plane strain are assumed in the domain below Γ_L . To visualize the problem refer to figure 1.1

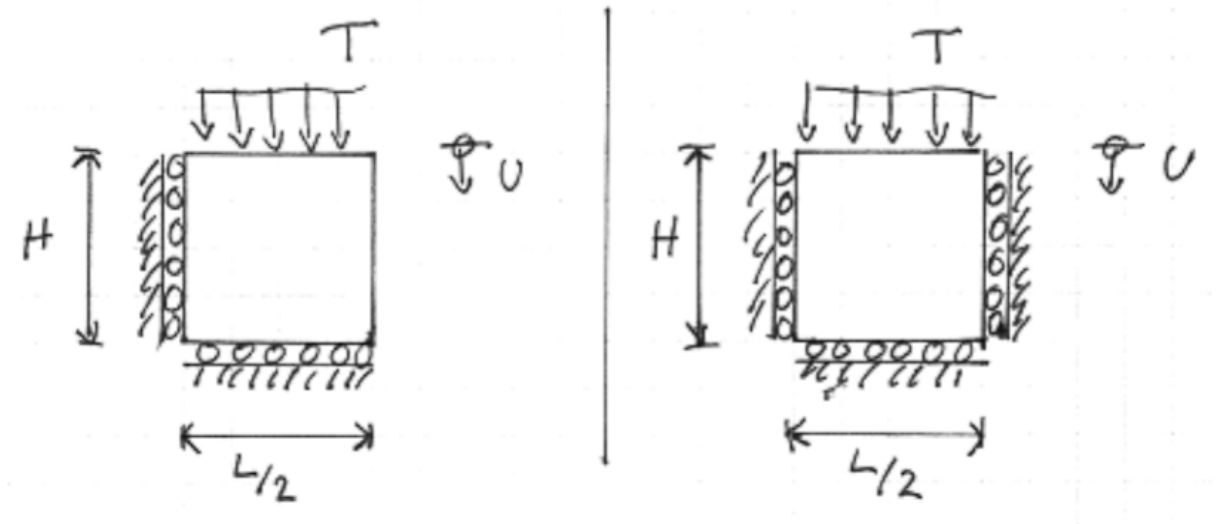


Figure 1.1: Rectangular Domain for Task 1

1.2 Computing relation between T and U

To obtain the analytical approximation for the traction-displacement relation two extreme condition are considered over the rectangular domain beneath Γ_L as showed in figure 1.1, such that

$$T = -\bar{t}_2 = -\sigma_{22}, \quad U = -\bar{u}_2 = -\epsilon_{22}H$$

1.2.1 Case I: $\epsilon_{11} = 0$

In this case both the vertical faces prevent horizontal displacement, so the lateral strain is fixed to

$$\epsilon_{11} = 0$$

The 2D constitutive relation under the plane strain can be written as

$$\begin{bmatrix} \sigma_{11} \\ \sigma_{22} \end{bmatrix} = \frac{E}{(1+\nu)(1-2\nu)} \begin{bmatrix} 1-\nu & \nu \\ \nu & 1-\nu \end{bmatrix} \begin{bmatrix} \epsilon_{11} \\ \epsilon_{22} \end{bmatrix} \quad (1.1)$$

After, applying the condition of $\epsilon_{11} = 0$ in equation 1.1 the second row of the matrix can be written as

$$\sigma_{22} = \frac{E}{(1+\nu)(1-2\nu)} (1-\nu) \epsilon_{22}$$

which after modification can be written as

$$T = \frac{E}{(1+\nu)(1-2\nu)} \frac{(1-\nu)U}{H} \quad (1.2)$$

1.2.2 Case II: $\sigma_{11} = 0$

In this case the domain is bounded by the left and the bottom boundaries in x and y direction respectively and the vertical face can be assumed as traction free resulting in

$$\sigma_{11} = 0$$

The 2D constitutive relation from equation 1.1 can be modified and written as

$$\begin{bmatrix} \epsilon_{11} \\ \epsilon_{22} \end{bmatrix} = \frac{1-\nu^2}{E} \begin{bmatrix} 1 & -\nu/(1-\nu) \\ -\nu/(1-\nu) & 1 \end{bmatrix} \begin{bmatrix} \sigma_{11} \\ \sigma_{22} \end{bmatrix} \quad (1.3)$$

After applying the condition of $\sigma_{11} = 0$ in equation 1.3 the second row of the matrix can be written as

$$\epsilon_{22} = \frac{1-\nu^2}{E} \sigma_{22}$$

which after modification can be written as

$$T = \frac{E}{(1-\nu^2)} \frac{U}{H} \quad (1.4)$$

Task 2

2.1 Problem Description

In this task a two-dimensional domain is considered as shown in figure 2.1 which is modeled under the plane strain condition and subjected to symmetric boundary condition on the left and bottom edges.

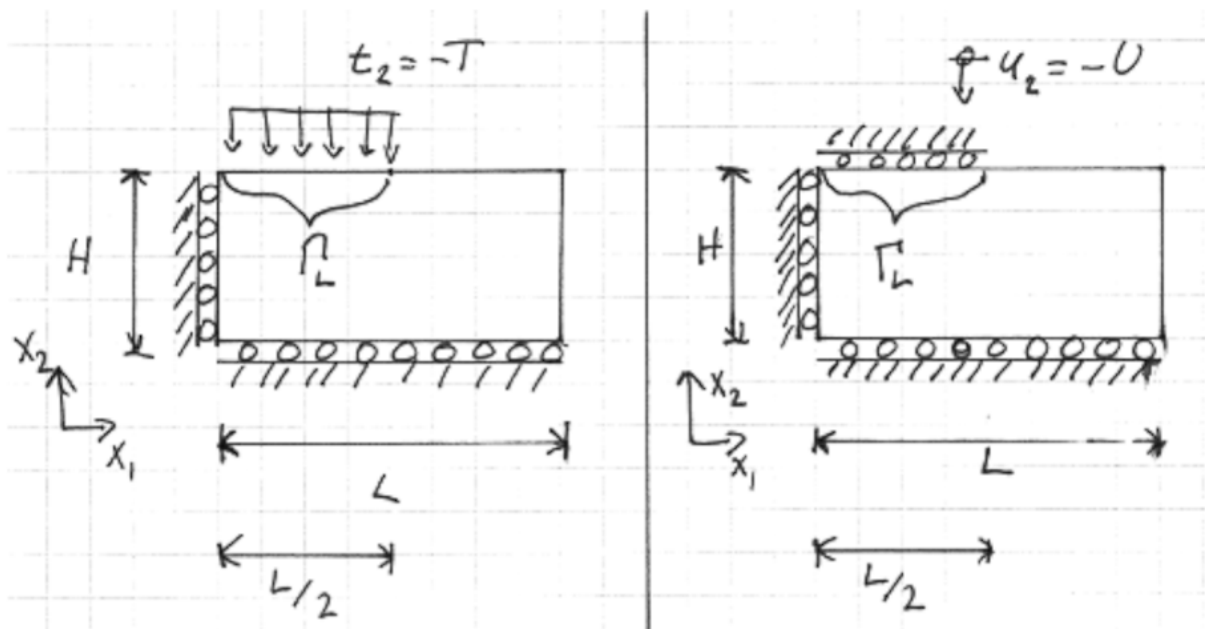


Figure 2.1: Rectangular Domain for Task 2

The main aim of this task is investigate the effect of the varying values of ν approaching the limit of $\nu \rightarrow 0.5$. Results from scenario 1 and scenario 2 will also be compared with the analytical limits computed from task 1.

To make sure the results are accurate, mesh convergence study is also conducted for $\bar{t}_2$ and $\bar{u}_2$ with mesh refinement done for each value of ν .

Pressure distribution will also be plotted for varying values of ν for both the scenarios.

2.2 Methodology

The numerical investigation is conducted using Constant Strain Triangle under the plane strain condition. The domain uses the symmetric boundary condition along the left and bottom boundary edges, and top edge under Γ_L is subject to prescribed vertical traction (*CASE I*) or prescribed vertical displacement (*CASE II*). The traction

controlled case applies a uniform loading of $T = 10^6 \text{ Pa}$, while the displacement controlled case applies a prescribed vertical displacement of $U = 10^{-7} \text{ m}$. CST elements are chosen to study how the element behaves as it reaches incompressibility. The material model is linear elastic with a fixed elastic modulus of $E = 200 \text{ GPa}$ (which is an assumed value in this assignment), while the Poisson's ratio is varied over 0.3, 0.45, 0.48, 0.49, 0.499, 0.4999 to examine how the response evolves as $\nu \rightarrow 0.5$.

The computational implementation is carried out by partly using CALFEM Toolbox for element stiffness matrix and computing pressure distribution over the domain integrated with the original matlab code, which can be seen in Appendix A. For each case and each value of Poisson's ratio, the finite element solution is obtained and used to compute two key quantities.

1. CASE I (Uniform Traction Load): Average Displacement on the top left edge i.e. Γ_L is computed.

$$\bar{u}_2 = \frac{1}{\Gamma_L} \int_{\Gamma_L} e_2 \cdot u \, d\Gamma$$

2. CASE II (Prescribed Displacement Load): Average Traction on the top left edge i.e. Γ_L is computed.

$$\bar{t}_2 = \frac{1}{\Gamma_L} \int_{\Gamma_L} e_2 \cdot t \, d\Gamma$$

In addition pressure within each element is evaluated using $p = -\sigma_m$ and also plotted for each value of ν . These numerical result is then compared to the analytical limiting cases derived in Task 1, which provide upper and lower bound within which the numerical solution should exist.

To ensure that the results are not significantly influenced by the size of the mesh, a mesh convergence study is also conducted for each value of ν , where the convergence behavior of $\bar{t}_2$ and $\bar{u}_2$ is studied for *CASE I* and *CASE II*.

2.3 Result and Analysis

This section presents the numerical results obtained for both loading cases and for varying values of Poisson's ratio ν . The influence of mesh refinement has also been analyzed in this section

2.3.1 Effect of Poisson's ratio

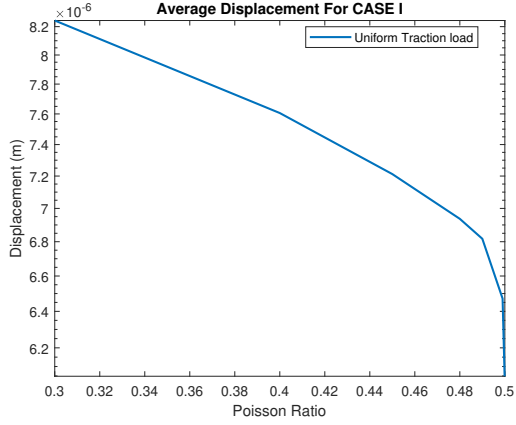


Figure 2.2: Uniform Traction Load Case

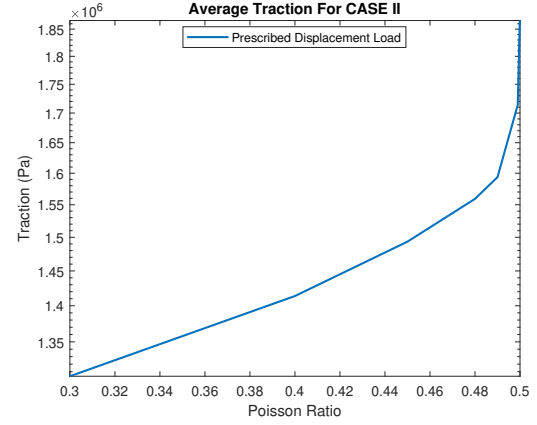
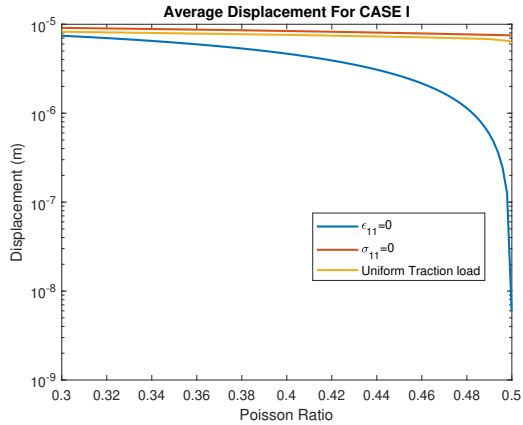
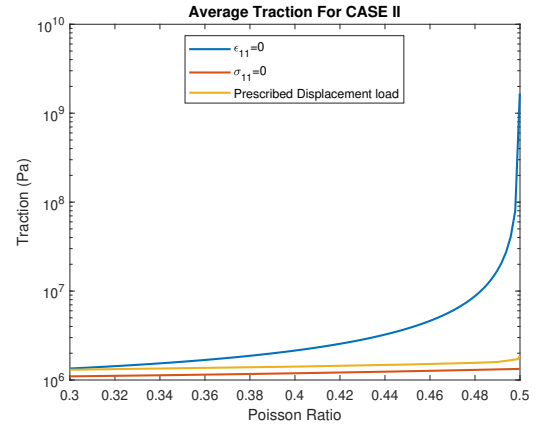


Figure 2.3: Pres. Displacement Load Case

Figures 2.2 and 2.3 show the variation of the average displacement in *CASE I* and the average traction in *CASE II* as ν increases from 0.3 towards 0.5. As the ν approaches 0.5, both the responses steepen due to the formulations tendency to reach singularity near incompressibility.



(a) Uniform Traction Load Case

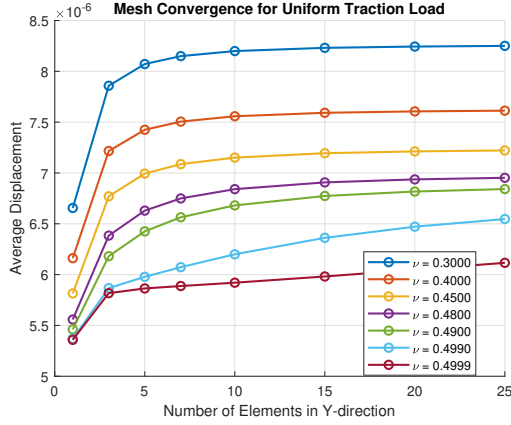


(b) Pres. Displacement Load Case

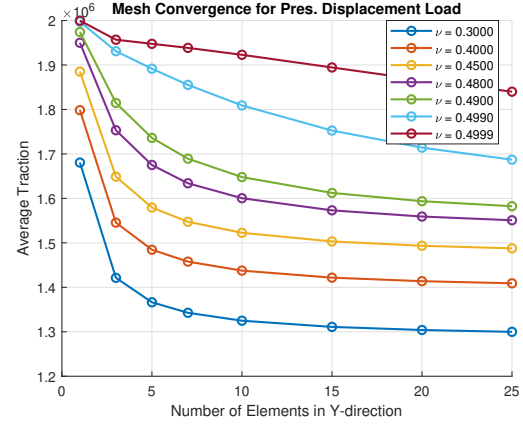
Figure 2.4: CST Element routine compared with two analytical limiting cases

However, even at $\nu = 0.499$, the numerical values in both cases remains within bounds, since the analytical solution used in Task 1 also exhibits singular behavior as $\nu \rightarrow 0.5$ as shown in figures 2.4a and 2.4b. Hence, even near incompressibility, the solution does not exceed the theoretical bounds even though the solution is unstable.

2.3.2 Mesh Convergence Study



(a) Uniform Traction Load Case



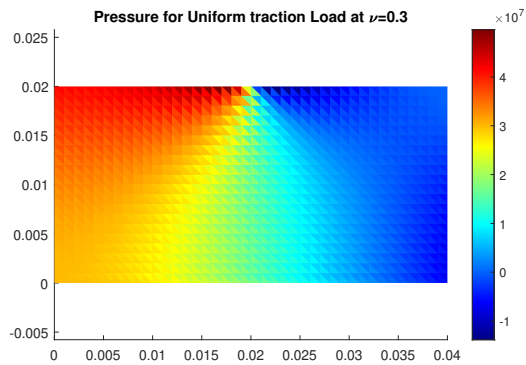
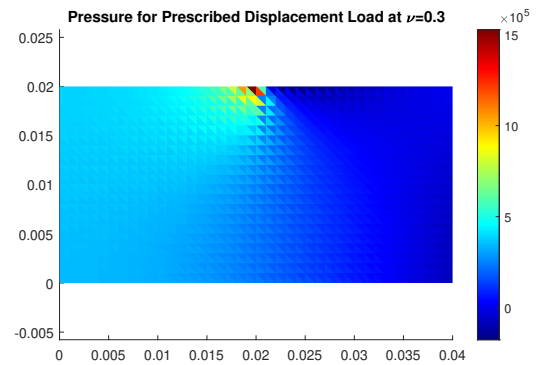
(b) Pres. Displacement Load Case

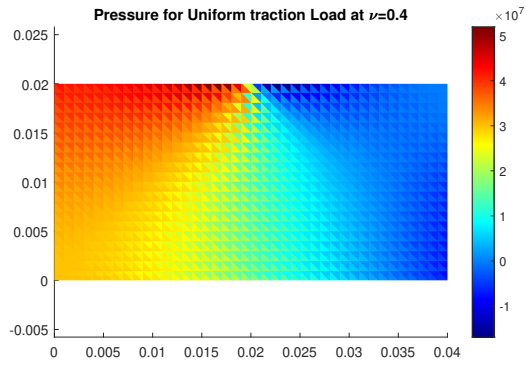
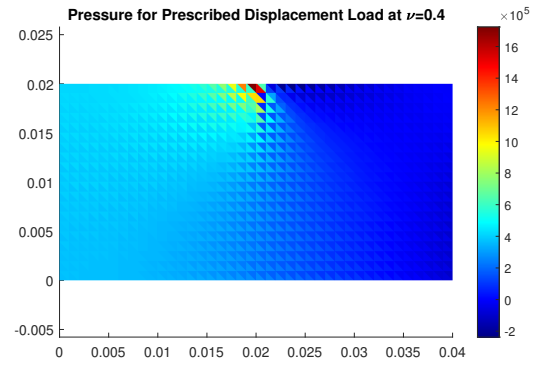
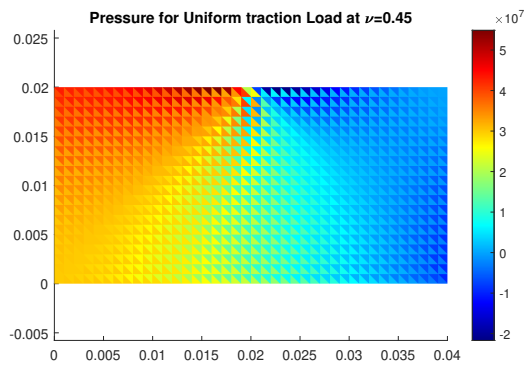
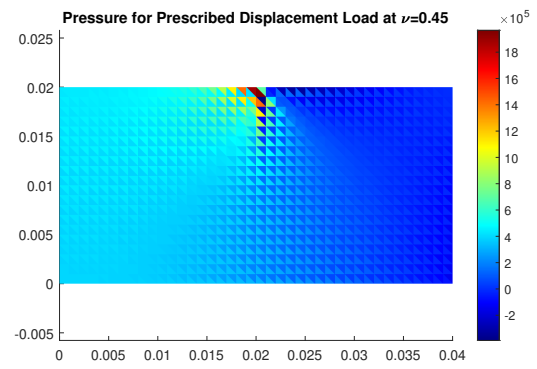
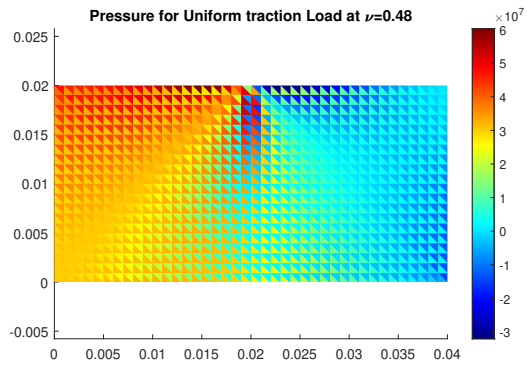
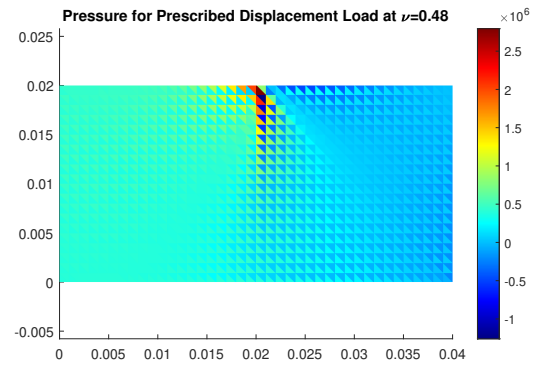
Figure 2.5: Mesh Convergence Study

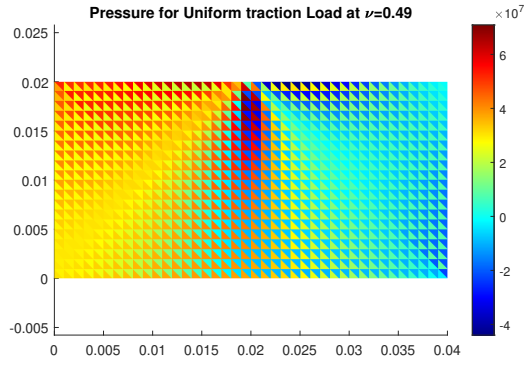
Mesh refinement was performed for each value of Poisson's ratio, and the convergence of $\bar{u}_2$ and $\bar{t}_2$ is shown in figures 2.5a and 2.5b. For moderate values of ν , the results converge quickly as the mesh is refined. But for values of ν near incompressibility, the solution convergence becomes slower but the value still keeps rising potentially because of solutions tendency to reach singularity.

2.3.3 Pressure Distribution

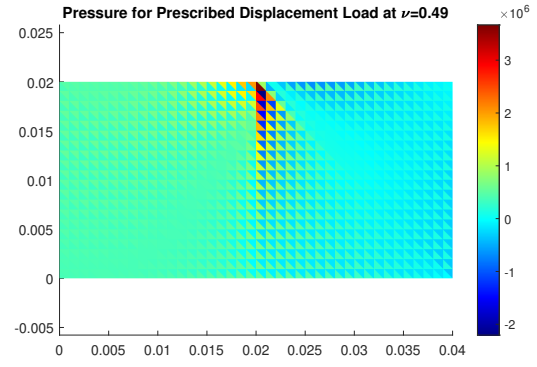
The pressure distribution for each Poisson's ratio is shown in figures below. For small ν values, the pressure variation is smooth and physically reasonable. But as the value of ν increases, the magnitude of maximum pressure grows and the field becomes much more irregular and a checkerboarding pattern appears.

Figure 2.6: Pressure Distribution for 2 Cases for different values of ν (a) CASE I: $\nu = 0.3$ (b) CASE II: $\nu = 0.3$

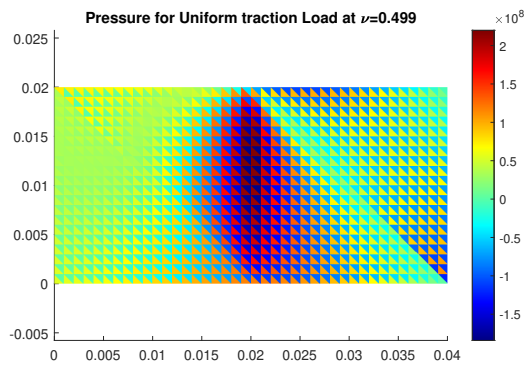
(c) CASE I: $\nu = 0.4$ (d) CASE II: $\nu = 0.4$ (e) CASE I: $\nu = 0.45$ (f) CASE II: $\nu = 0.45$ (g) CASE I: $\nu = 0.48$ (h) CASE II: $\nu = 0.48$



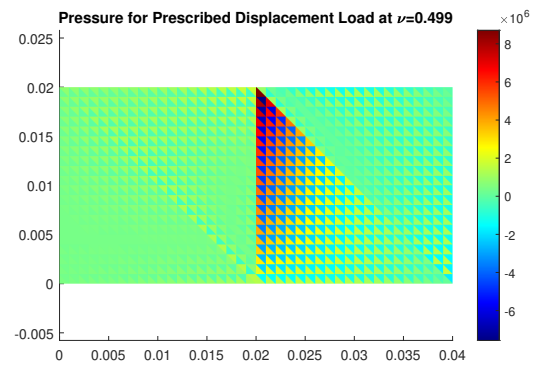
(i) CASE I: $\nu = 0.49$



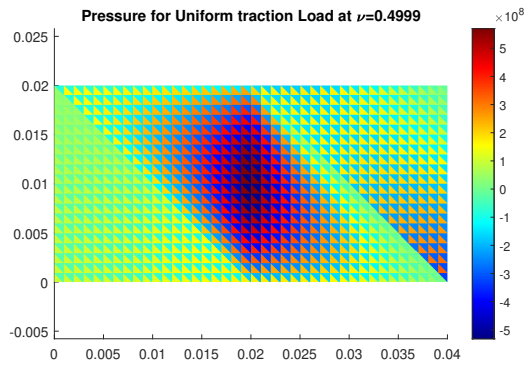
(j) CASE II: $\nu = 0.49$



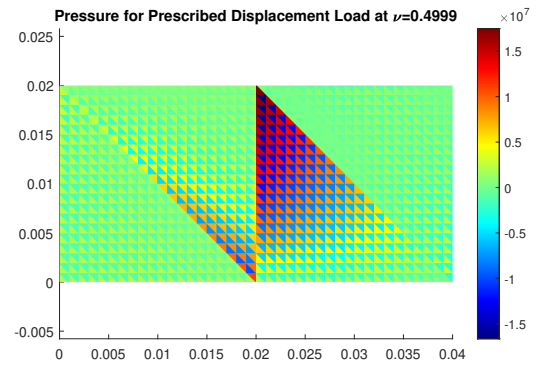
(k) CASE I: $\nu = 0.499$



(l) CASE II: $\nu = 0.499$



(m) CASE I: $\nu = 0.4999$



(n) CASE II: $\nu = 0.4999$

3

Task 3

3.1 Problem Description

Task 3 focuses on evaluating a Taylor-Hood mixed finite element for nearly incompressible elasticity. The element uses quadratic shape function for displacement combined with linear shape function for the pressure nodes, which provides stability as $\nu \rightarrow 0.5$.

The same two loading cases as in task 2 are analyzed:

1. CASE I : Uniform Traction Load $t = -Te_2$ on Γ_L is applied to compute resulting average vertical displacement $\bar{u}_2$.
2. CASE II: Uniform Displacement $u = -Ue_2$ is applied on Γ_L to determine the reaction traction $\bar{t}_2$.

The Taylor-Hood element is implemented for several Poisson's ratio approaching incompressibility, including $\nu = 0.5$. To make sure the results are accurate, mesh convergence study is also conducted for $\bar{t}_2$ and $\bar{u}_2$ with mesh refinement done for each value of ν . The pressure distribution is also compared with the CST element results from Task 2. The goal is to demonstrate the improved behavior of the mixed formulation in producing smooth, and meaningful pressure fields compared to checkerboarding pattern observed in previous as ν reaches incompressibility.

3.2 Methodology

For task 3, a mixed finite element formulation is implemented using Taylor-Hood triangular element. Here the displacement field is approximated with quadratic (6-node) shape functions and pressure field with linear (3-node) shape functions. This element satisfies the inf-sup stability condition and hence suitable for nearly incompressible and fully incompressible elasticity.

The simulation is carried out for the same two loading cases as defined problem formulation. A sequence of uniform meshes with increasing refinement was generated and the Taylor-Hood dofs are assembled using custom element routine. For each mesh, the mixed displacement-pressure system was solved under both loading cases.

$$\begin{bmatrix} K_{uu} & K_{up} \\ K_{pu} & K_{pp} \end{bmatrix} \begin{bmatrix} a_{udof} \\ a_{pdof} \end{bmatrix} = \begin{bmatrix} f \\ 0 \end{bmatrix}$$

The Poisson's ratio was varied over $\nu = \{0.3, 0.4, 0.45, 0.48, 0.49, 0.5\}$ to study the performance of the Taylor-Hood formulation as the material approaches incompressibility.

For each combination of mesh and ν , average displacement and average traction has been evaluated to study the mesh convergence and to ensure sufficient refinement of mesh so that we get accurate results without loss of excess of computational resources.

3.3 Result and Analysis

This section presents the numerical results obtained using the Taylor-Hood mixed formulation for the two loading case.

3.3.1 Uniform Traction Load Case

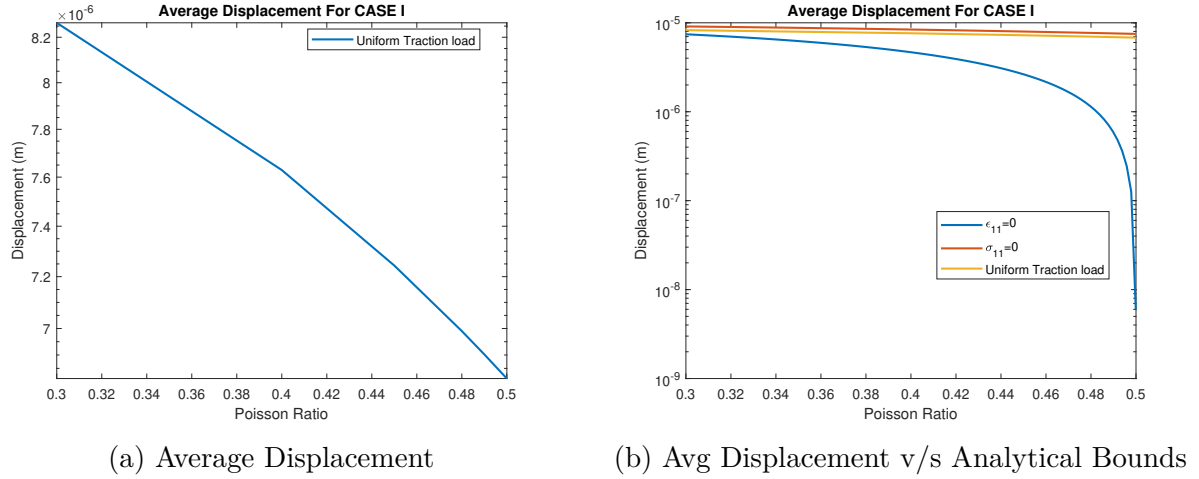


Figure 3.1: Uniform Traction Load Case

Figure 3.1a shows the computed average vertical displacement $\bar{u}_2$ for the prescribed traction case across different values of Poisson's ratio. The Taylor-Hood element produces the downward trend of displacement as expected for values of ν reaching incompressibility, including the ones that are near incompressibility. This element routine has produced the displacement that stays within the bounds calculated from Task 1, as can be seen in 3.1b. But the key observation was that even at $\nu = 0.5$ the result remains stable and do not exhibit any kind of singularity trend as can be seen from figure 2.2 of task 2.

3.3.2 Prescribed Displacement Load Case

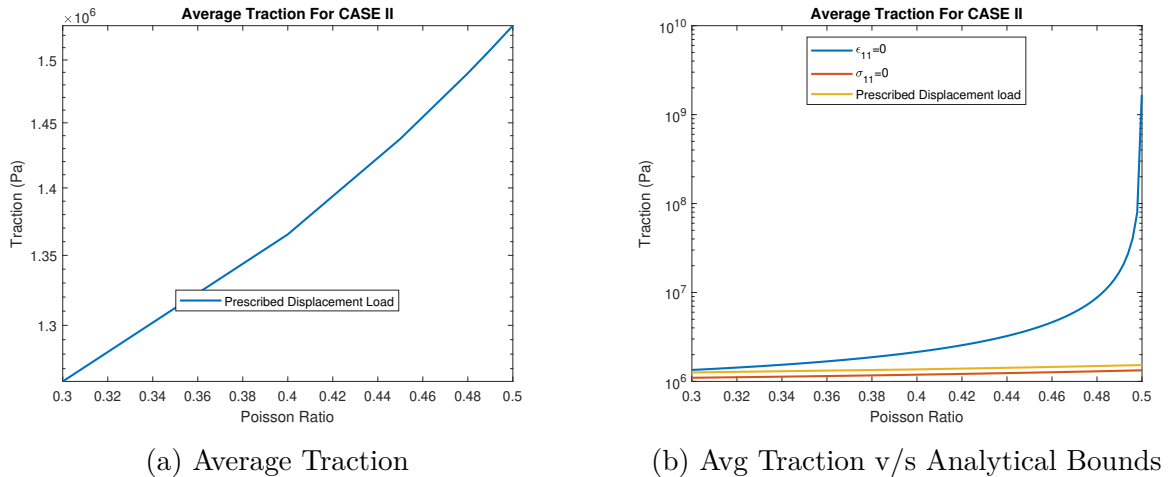


Figure 3.2: Prescribed Displacement Load Case

Figure 3.2a shows the computed average vertical traction $\bar{t}_2$ for the prescribed displacement load case across different values of Poisson's ratio. The Taylor-hood element produces the upward trend of traction as expected for values of ν reaching incompressibility. This element routine has produced the traction that stay within the analytical bounds calculated from task 1, as can be seen in figure 3.2b. But the key observation was that even at $\nu = 0.5$ the result remains stable and do not exhibit any kind of singularity trend as seen from figure 2.3 of task 2.

3.3.3 Mesh Convergence Study

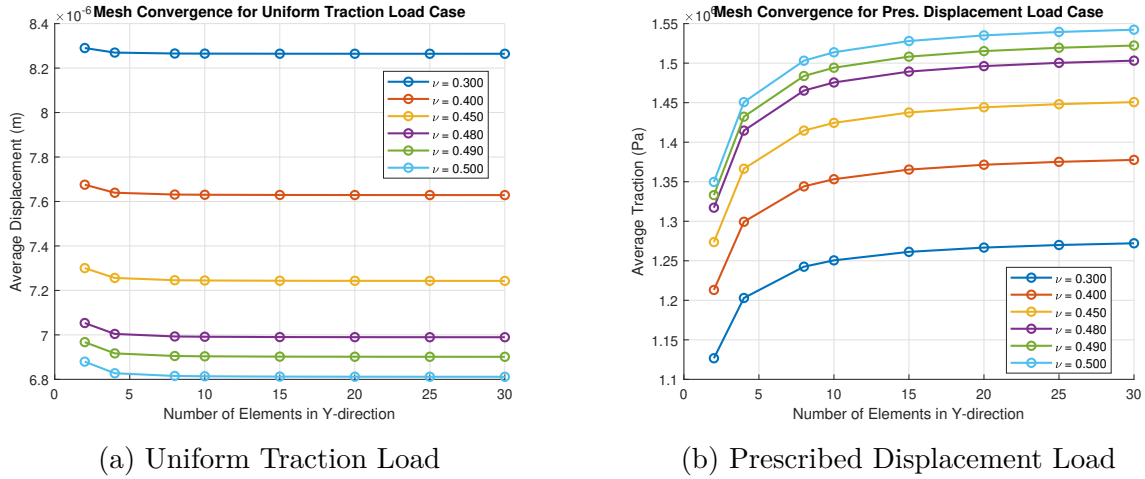
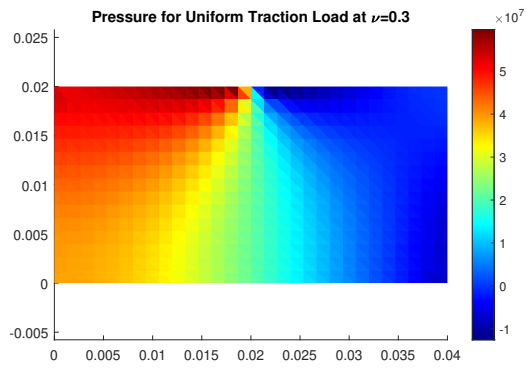
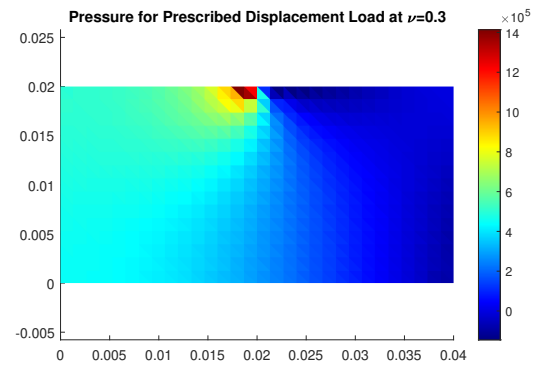
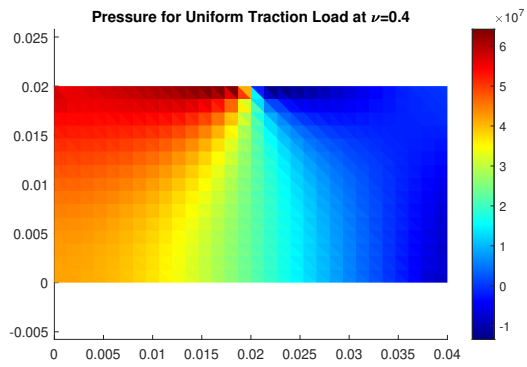
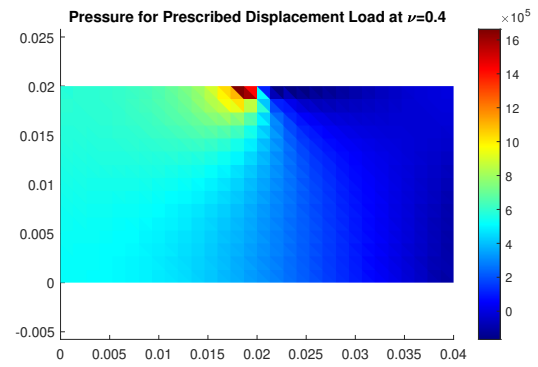
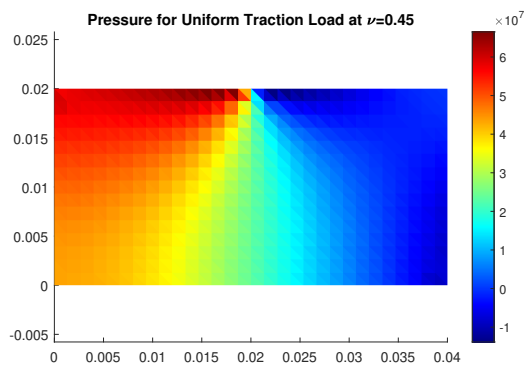
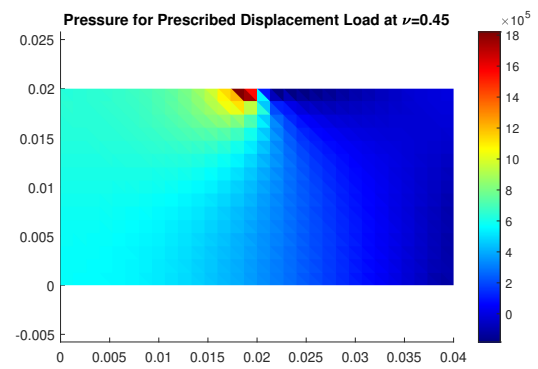


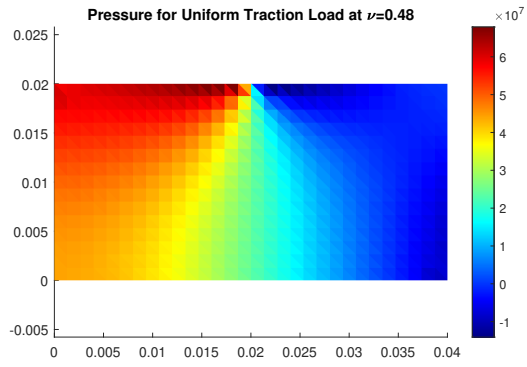
Figure 3.3: Mesh Convergence Study

Mesh refinement was performed for each value of Poisson's ratio and the convergence of $\bar{u}_2$ and $\bar{t}_2$ is shown in figures 3.3a and 3.3b. For all values of ν , the results converge quickly as the mesh is refined. The values of ν near incompressibility also converges quickly compared to its counterpart in task 2.

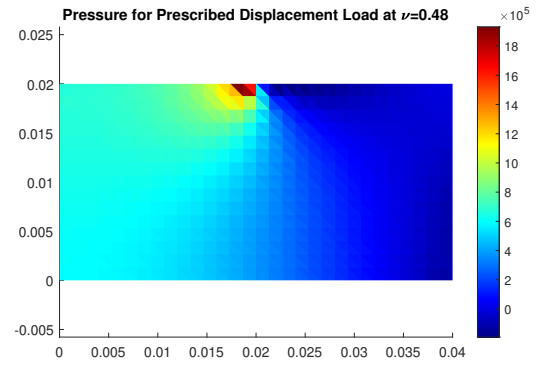
3.3.4 Pressure Distribution

The pressure distribution for each Poisson's ratio is shown in figures below. For all ν , the computed pressure remains smooth and physically reasonable. Importantly, at $\nu = 0.5$, Taylor-Hood elements maintain stability and yields a continuous pressure distribution, compared to task 2 where checkerboarding pattern was observed for near incompressibility value of ν . Also the pressure plots for smaller ν values, for example $\nu = 0.3$, the pressure plot contours matches with each other which also confirms the correct implementation of Taylor-Hood element routine, because for lower values of ν the solution is stable in both task 2 and task 3.

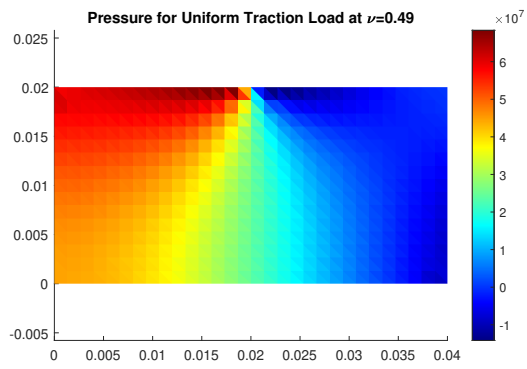
Figure 3.4: Pressure Distribution for 2 Cases for different values of ν (a) CASE I: $\nu = 0.3$ (b) CASE II: $\nu = 0.3$ (c) CASE I: $\nu = 0.4$ (d) CASE II: $\nu = 0.4$ (e) CASE I: $\nu = 0.45$ (f) CASE II: $\nu = 0.45$



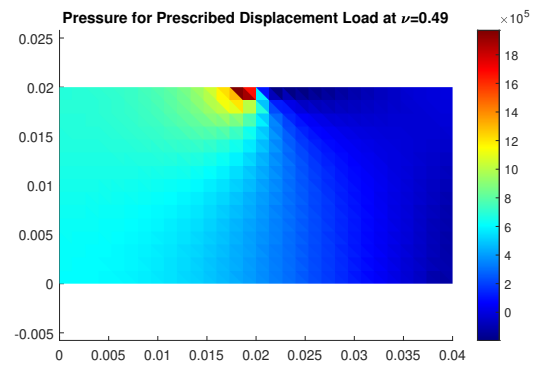
(g) CASE I: $\nu = 0.48$



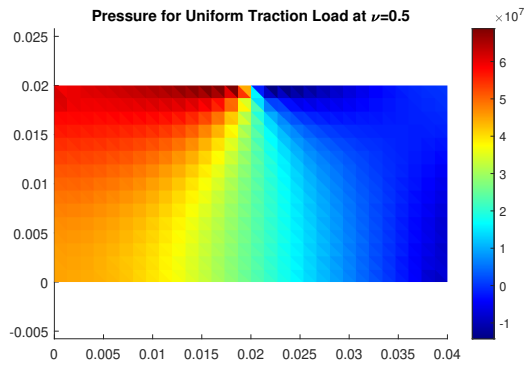
(h) CASE II: $\nu = 0.48$



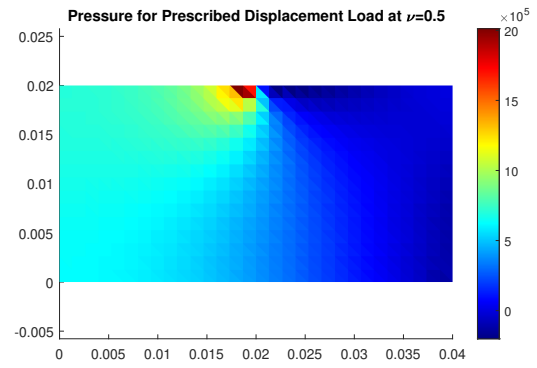
(i) CASE I: $\nu = 0.49$



(j) CASE II: $\nu = 0.49$



(k) CASE I: $\nu = 0.5$



(l) CASE II: $\nu = 0.5$

4

Conclusion

The analytical solution in Task 1 establishes the limiting bounds for the domain for different values of ν ranging from $0.3 \rightarrow 0.4999$. Using these as reference, the numerical studies in Task 2 and Task 3 demonstrated different behavior of the two finite element formulation. The CST element showed that as ν was approaching incompressibility, the pressure field started to show checkerboard patterns and the average displacement and average traction showing trends to reach singularity.

In contrast, the Taylor-Hood mixed formulation remained stable for all values of ν , including $\nu = 0.5$ (Incompressible Case). It produced smooth pressure distribution, a stable average displacement and traction responses with a clear convergence under mesh refinement. Overall, the results confirm that the Taylor-Hood element is well suited for incompressible elastic response

A

Appendix: Task 2 Code List

A.1 Main Code

```
1 clear all
2 close all
3 clc
4 %%
5
6 ny_list = [1,3,5,7,10,15,20,25];
7 nu0 = [0.3,0.4,0.45,0.48,0.49,0.499,0.4999];
8
9 E = 200e9; % Pa
10 thickness = 1; % m
11 H = 20e-3; %m
12
13 %% Load Cases
14
15 %Case 1
16 Tval = 1e8; % Pa
17 ueval = zeros(size(nu0,2),size(ny_list,2));
18 %Case 2
19
20 Uval = 1e-7; %m
21 Teval = zeros(size(nu0,2),size(ny_list,2));
22 %% Problem 1
23
24
25 mu = linspace(0.3,0.4999,100);
26 U_1 = zeros(size(mu,2),1);
27 U_2 = zeros(size(mu,2),1);
28 T_1 = zeros(size(mu,2),1);
29 T_2 = zeros(size(mu,2),1);
30
31 for i = 1:size(mu,2)
32
33     U_1(i,1) = Tval/((E*(1-mu(i)))/((1+mu(i))*(1-2*mu(i))*H));
34     U_2(i,1) = Tval/((E)/((1-mu(i)^2)*H));
35     T_1(i,1) = Uval*((E*(1-mu(i)))/((1+mu(i))*(1-2*mu(i))*H));
36     T_2(i,1) = Uval*((E)/((1-mu(i)^2)*H));
37 end
38
39 %% Problem 2
40
41 for iny = 1:size(ny_list,2)
42
43     ny = ny_list(iny);
```

```

44 order = 1;
45 [coords, edges, triangle] = mesh(ny,order,false);
46
47 Ex = coords(1,:);
48 Ey = coords(2,:);
49
50
51 nnodes = size(coords,2);
52 nelem = size(triangle,2);
53 Edof = zeros(nelem, size(triangle,1)*2+1);
54
55 for i = 1:nelem
56
57     dofx = 2*triangle(:,i)-1;
58     dofy = 2*triangle(:,i);
59     Edof(i,1)=i;
60     Edof(i,2:2:end) = dofx;
61     Edof(i,3:2:end) = dofy;
62
63 end
64 %% Material Properties
65
66
67 ptype = 2;
68 ep = [ptype thickness];
69
70 ndof = 2*nnodes;
71
72 %% Case I - Problem 2
73
74 for inu = 1:size(nu0,2)
75
76     nu = nu0(inu);
77
78     D = hooke(ptype,E,nu);
79
80     K = sparse(ndof,ndof);
81     F = zeros(ndof,1);
82     bound_id = 4;
83
84     for ielem = 1:nelem
85
86         enodes = triangle(:,ielem);
87         ex = coords(1,enodes);
88         ey = coords(2,enodes);
89         [Ke,fe] = plante(ex,ey,ep,D);
90
91         edof = zeros(1,6);
92
93         for k = 1:3
94
95             n = enodes(k);
96             edof(2*k-1:2*k) = [2*n-1,2*n];
97         end
98
99         K(edof,edof) = K(edof,edof)+Ke;
100         F(edof) = F(edof)+fe;
101

```

```

102     end
103
104     [funit,gamma_L] = Loadvector_interpolation(coords,edges,bound_id);
105
106     F_T = F + (-Tval)*funit;
107
108     fixed_dofs = [];
109     left_edge = find(edges(end,')==5);
110
111     for le = 1:size(left_edge,2)
112         n1 = edges(1,left_edge(1,le));
113         n2 = edges(2,left_edge(1,le));
114
115         fixed_dofs = [fixed_dofs,2*n1-1,2*n2-1];
116     end
117
118     bottom_edge = find(edges(end,')==1);
119
120     for be = 1:size(bottom_edge,2)
121         n1 = edges(1,bottom_edge(1,be));
122         n2 = edges(2,bottom_edge(1,be));
123
124         fixed_dofs = [fixed_dofs,2*n1,2*n2];
125     end
126
127     fdof = unique(fixed_dofs);
128     free_dof = setdiff(1:ndof,fdof);
129
130     u= zeros(ndof,1);
131
132     K_free = K(free_dof,free_dof);
133     F_free = F_T(free_dof);
134
135     u(free_dof) = K_free\F_free;
136
137     ueval(inu,iny) = -(1/(gamma_L))*(funit'*u);
138     %
139
140 end
141
142
143 %% Case II - Problem 2
144
145 for inu = 1:size(nu0,2)
146
147     nu = nu0(inu);
148
149     D = hooke(ptype,E,nu);
150
151     K = sparse(ndof,ndof);
152     F = zeros(ndof,1);
153     bound_id = 4;
154
155     for ielem = 1:nelem
156         enodes = triangle(:,ielem);
157         ex = coords(1,enodes);
158         ey = coords(2,enodes);
159         [Ke,fe] = plante(ex,ey,ep,D);

```

```

160
161     edof = zeros(1,6);
162
163     for k = 1:3
164
165         n = enodes(k);
166         edof(2*k-1:2*k) = [2*n-1,2*n];
167     end
168
169     K(edof,edof) = K(edof,edof)+Ke;
170     F(edof) = F(edof)+fe;
171
172 end
173
174 fixed_dofs2 = [];
175 left_edge = find(edges(end,')==5);
176
177 for le = 1:size(left_edge,2)
178     n1 = edges(1,left_edge(1,le));
179     n2 = edges(2,left_edge(1,le));
180
181     fixed_dofs2 = [fixed_dofs2,2*n1-1,2*n2-1];
182 end
183
184 bottom_edge = find(edges(end,')==1);
185
186 for be = 1:size(bottom_edge,2)
187     n1 = edges(1,bottom_edge(1,be));
188     n2 = edges(2,bottom_edge(1,be));
189
190     fixed_dofs2 = [fixed_dofs2,2*n1,2*n2];
191 end
192
193 top_left_edge = find(edges(end,')==4);
194 pres_dofs = [];
195 for tle = 1:size(top_left_edge,2)
196     n1 = edges(1,top_left_edge(1,tle));
197     n2 = edges(2,top_left_edge(1,tle));
198
199     fixed_dofs2 = [fixed_dofs2,2*n1,2*n2];
200     pres_dofs = [pres_dofs, 2*n1, 2*n2];
201 end
202
203 fixdof2 = unique(fixed_dofs2);
204 pdof = unique(pres_dofs);
205 free_dof2 = setdiff(1:ndof,fixdof2);
206
207 u2 = zeros(ndof,1);
208 u2(pdof) = -Uval;
209 K_f = K(free_dof2,free_dof2);
210 F_f = F(free_dof2,1);
211 K_fc = K(free_dof2,fixdof2);
212
213 u_f = K_f \ (F_f - K_fc*u2(fixdof2));
214
215 u2(free_dof2) = u_f;
216
217 RF = K*u2;

```

```

218
219     T = 0;
220
221     for i = 1: size(pdof,2)
222         T = T+ RF(pdof(i));
223     end
224
225     Teval(inu,iny) = -T/gamma_L ;
226
227 end
228
229
230 end
231
232 figure; hold on; grid on;
233 colors = lines(numel(nu0));
234
235 for j = 1:numel(nu0)
236     plot(ny_list, ueval(j,:), '-o', ...
237         'Color', colors(j,:), 'LineWidth', 1.5, 'DisplayName', sprintf('
238         \\nu = %.4f', nu0(j)));
239 end
240 set(gca, 'FontSize');
241 xlabel('Number of Elements in Y-direction');
242 ylabel('Average Displacement');
243 title('Mesh Convergence for Uniform Traction Load');
244 legend(Location='best');
245
246 figure; hold on; grid on;
247 colors = lines(numel(nu0));
248
249 for j = 1:numel(nu0)
250     plot(ny_list, Teval(j,:), '-o', ...
251         'Color', colors(j,:), 'LineWidth', 1.5, 'DisplayName', sprintf('
252         \\nu = %.4f', nu0(j)));
253 end
254 set(gca, 'FontSize');
255 xlabel('Number of Elements in Y-direction');
256 ylabel('Average Traction');
257 title('Mesh Convergence for Pres. Displacement Load');
258 legend(Location='best');
259
260
261
262
263 %% Pressure Distribution
264
265 ny = 20;
266 order = 1;
267 [coords, edges, triangle] = mesh(ny,order,false);
268
269 Ex = coords(1,:);
270 Ey = coords(2,:);
271
272 ueval_c = zeros(size(nu0,2),1);
273 Teval_c = zeros(size(nu0,2),1);

```

```

274
275 nnodes = size(coords,2);
276 nelem = size(triangle,2);
277 Edof = zeros(nelem, size(triangle,1)*2+1);
278
279 for i = 1:nelem
280
281     dofx = 2*triangle(:,i)-1;
282     dofy = 2*triangle(:,i);
283     Edof(i,1)=i;
284     Edof(i,2:2:end) = dofx;
285     Edof(i,3:2:end) = dofy;
286
287 end
288 %% Material Properties
289
290
291 ptype = 2;
292 ep = [ptype thickness];
293
294 ndof = 2*nnodes;
295
296 %% Case I - Problem 2
297
298 for inu = 1:size(nu0,2)
299
300     nu = nu0(inu);
301
302     D = hooke(ptype,E,nu);
303
304     K = sparse(ndof,ndof);
305     F = zeros(ndof,1);
306     bound_id = 4;
307
308     for ielem = 1:nelem
309
310         enodes = triangle(:,ielem);
311         ex = coords(1,enodes);
312         ey = coords(2,enodes);
313         [Ke,fe] = plante(ex,ey,ep,D);
314
315         edof = zeros(1,6);
316
317         for k = 1:3
318
319             n = enodes(k);
320             edof(2*k-1:2*k) = [2*n-1,2*n];
321         end
322
323         K(edof,edof) = K(edof,edof)+Ke;
324         F(edof) = F(edof)+fe;
325
326     end
327
328     [funit,gamma_L] = Loadvector_interpolation(coords,edges,bound_id);
329
330     F_T = F + (-Tval)*funit;
331

```

```

332     fixed_dofs = [];
333     left_edge = find(edges(end,:) == 5);
334
335     for le = 1:size(left_edge,2)
336         n1 = edges(1,left_edge(1,le));
337         n2 = edges(2,left_edge(1,le));
338
339         fixed_dofs = [fixed_dofs, 2*n1-1, 2*n2-1];
340     end
341
342     bottom_edge = find(edges(end,:) == 1);
343
344     for be = 1:size(bottom_edge,2)
345         n1 = edges(1,bottom_edge(1,be));
346         n2 = edges(2,bottom_edge(1,be));
347
348         fixed_dofs = [fixed_dofs, 2*n1, 2*n2];
349     end
350
351     fdof = unique(fixed_dofs);
352     free_dof = setdiff(1:ndof, fdof);
353
354     u = zeros(ndof,1);
355
356     K_free = K(free_dof, free_dof);
357     F_free = F_T(free_dof);
358
359     u(free_dof) = K_free \ F_free;
360
361     ueval_c(inu,1) = -(1/(gamma_L))*(funit'*u);
362     %
363     p1 = zeros(nelem,1);
364
365     for ie = 1:nelem
366
367         enodes = triangle(:,ie);
368         ex1 = coords(1,enodes);
369         ey1 = coords(2,enodes);
370
371         ed = zeros(6,1);
372
373         for k = 1:size(ed,1)/2
374
375             n = enodes(k);
376             ed(2*k-1) = u(2*n-1);
377             ed(2*k) = u(2*n);
378
379         end
380
381         [es,et] = plants(ex1,ey1,ep,D,ed');
382         p1(ie) = (-1/3)*(es(1)+es(2));
383
384     end
385
386
387     figure; hold on; axis equal; view(2);
388     patch('Faces', triangle, 'Vertices', coords, ...
389           'FaceColor', 'flat', 'CData', p1, 'EdgeColor', 'none');

```

```

390     colormap(jet); colorbar;
391     title(['Pressure for Uniform traction Load at \nu=',num2str(nu)]);
392
393 end
394
395
396
397 figure
398 semilogy(mu,U_1,LineWidth=1.5,DisplayName='\epsilon_{11}=0')
399 hold on
400 semilogy(mu,U_2,LineWidth=1.5,DisplayName='\sigma_{11}=0')
401 semilogy(nu0,ueval_c, LineWidth=1.5,DisplayName='Uniform Traction load')
402 hold off
403 set(gca, 'FontSize');
404 xlabel ('Poisson Ratio')
405 ylabel ('Displacement (m)')
406 title('Average Displacement For CASE I')
407 legend (Location="best")
408
409 figure
410
411 semilogy(nu0,ueval_c, LineWidth=1.5,DisplayName='Uniform Traction load')
412 set(gca, 'FontSize');
413 xlabel ('Poisson Ratio')
414 ylabel ('Displacement (m)')
415 title('Average Displacement For CASE I')
416 legend (Location="best")
417
418 %% Case II - Problem 2
419
420 for inu = 1:size(nu0,2)
421
422     nu = nu0(inu);
423
424     D = hooke(ptype,E,nu);
425
426     K = sparse(ndof,ndof);
427     F = zeros(ndof,1);
428     bound_id = 4;
429
430     for ielem = 1:nelem
431         enodes = triangle(:,ielem);
432         ex = coords(1,enodes);
433         ey = coords(2,enodes);
434         [Ke,fe] = plante(ex,ey,ep,D);
435
436         edof = zeros(1,6);
437
438         for k = 1:3
439
440             n = enodes(k);
441             edof(2*k-1:2*k) = [2*n-1,2*n];
442         end
443
444         K(edof,edof) = K(edof,edof)+Ke;
445         F(edof) = F(edof)+fe;
446
447     end

```



```

448
449     fixed_dofs2 = [];
450     left_edge = find(edges(end,:) == 5);
451
452     for le = 1:size(left_edge,2)
453         n1 = edges(1,left_edge(1,le));
454         n2 = edges(2,left_edge(1,le));
455
456         fixed_dofs2 = [fixed_dofs2, 2*n1-1, 2*n2-1];
457     end
458
459     bottom_edge = find(edges(end,:) == 1);
460
461     for be = 1:size(bottom_edge,2)
462         n1 = edges(1,bottom_edge(1,be));
463         n2 = edges(2,bottom_edge(1,be));
464
465         fixed_dofs2 = [fixed_dofs2, 2*n1, 2*n2];
466     end
467
468     top_left_edge = find(edges(end,:) == 4);
469     pres_dofs = [];
470     for tle = 1:size(top_left_edge,2)
471         n1 = edges(1,top_left_edge(1,tle));
472         n2 = edges(2,top_left_edge(1,tle));
473
474         fixed_dofs2 = [fixed_dofs2, 2*n1, 2*n2];
475         pres_dofs = [pres_dofs, 2*n1, 2*n2];
476     end
477
478     fixdof2 = unique(fixed_dofs2);
479     pdof = unique(pres_dofs);
480     free_dof2 = setdiff(1:ndof, fixdof2);
481
482     u2 = zeros(ndof,1);
483     u2(pdof) = -Uval;
484     K_f = K(free_dof2, free_dof2);
485     F_f = F(free_dof2, 1);
486     K_fc = K(free_dof2, fixdof2);
487
488     u_f = K_f \ (F_f - K_fc * u2(fixdof2));
489
490     u2(free_dof2) = u_f;
491
492     RF = K * u2;
493
494     T = 0;
495
496     for i = 1:size(pdof,2)
497         T = T + RF(pdof(i));
498     end
499
500     Teval_c(inu,1) = -T/gamma_L ;
501
502     p2 = zeros(nelem,1);
503
504     for ie = 1:nelem
505

```

```

506     enodes = triangle(:,ie);
507     ex2 = coords (1,enodes);
508     ey2 = coords (2,enodes);
509
510     ed = zeros(6,1);
511
512     for k = 1:size(ed,1)/2
513
514         n = enodes(k);
515         ed(2*k-1)= u2(2*n-1);
516         ed(2*k)= u2(2*n);
517
518     end
519
520     [es,et] = plants(ex2,ey2,ep,D,ed');
521     p2(ie) = (-1/3)*(es(1)+es(2));
522
523 end
524
525 figure; hold on; axis equal; view(2);
526 patch('Faces', triangle', 'Vertices', coords', ...
527     'FaceColor', 'flat', 'CData', p2, 'EdgeColor', 'none');
528 colormap(jet); colorbar;
529 title(['Pressure for Prescribed Displacement Load at \nu=',num2str(
530 nu)]);
531 end
532
533 %%
534 figure
535
536 semilogy(mu,T_1,LineWidth=1.5,DisplayName='\epsilon_{11}=0')
537 hold on
538 semilogy(mu,T_2,LineWidth=1.5,DisplayName='\sigma_{11}=0')
539 semilogy(nu0,Teval_c, LineWidth=1.5,DisplayName='Prescribed Displacement
540 load')
541 hold off
542 set(gca, 'FontSize');
543 xlabel ('Poisson Ratio')
544 ylabel ('Traction (Pa)')
545 title('Average Traction For CASE II')
546 legend (Location="best")
547
548 figure
549
550 semilogy(nu0,Teval_c, LineWidth=1.5,DisplayName='Prescribed Displacement
551 Load')
552 set(gca, 'FontSize');
553 xlabel ('Poisson Ratio')
554 ylabel ('Traction (Pa)')
555 title('Average Traction For CASE II')
556 legend (Location="best")

```

A.2 Load Interpolation For Linear Elements

```
1 function [funit,gamma_L] = Loadvector_interpolation(coords,edges,  
    bound_id)  
2  
3 nnodes = size(coords,2);  
4 ndof = nnodes*2;  
5 funit = zeros(ndof,1);  
6 gamma_L=0;  
7  
8 for ec = 1:size(edges,2)  
9  
10     id = edges(end,ec);  
11  
12     if id==bound_id  
13         n1 = edges(1,ec);  
14         n2 = edges(2,ec);  
15  
16         x1 = coords(:,n1);  
17         x2 = coords(:,n2);  
18         L = norm(x2-x1);  
19  
20         gamma_L = gamma_L+L;  
21  
22         funit(2*n1)=funit(2*n1)+L/2;  
23         funit(2*n2)=funit(2*n2)+L/2;  
24     end  
25  
26 end  
27  
28 end
```

B

Appendix: Task 3 Code List

B.1 Main Code

```
1 clear all
2 close all
3 clc
4
5 %%
6
7 ny_list = [2,4,8,10,15,20,25,30];
8 nu0 = [0.3,0.4,0.45,0.48,0.49,0.5];
9
10 E = 200e9; % Pa
11 t = 1; % m
12 H = 20e-3; %m
13
14 %% Case 1
15 Tval = 1e8; % Pa
16 ueval = zeros(size(nu0,2),size(ny_list,2));
17 ueval_c = zeros(size(nu0,2),1);
18 % Case 2
19
20 Uval = 1e-7; %m
21 Teval = zeros(size(nu0,2),size(ny_list,2));
22 Teval_c = zeros(size(nu0,2),1);
23 %% Problem 1
24
25
26 mu = linspace(0.3,0.4999,100);
27 U_1 = zeros(size(mu,2),1);
28 U_2 = zeros(size(mu,2),1);
29 T_1 = zeros(size(mu,2),1);
30 T_2 = zeros(size(mu,2),1);
31
32 for i = 1:size(mu,2)
33
34     U_1(i,1) = Tval/((E*(1-mu(i)))/((1+mu(i))*(1-2*mu(i))*H));
35     U_2(i,1) = Tval/((E)/((1-mu(i)^2)*H));
36     T_1(i,1) = Uval*((E*(1-mu(i)))/((1+mu(i))*(1-2*mu(i))*H));
37     T_2(i,1) = Uval*((E)/((1-mu(i)^2)*H));
38 end
39
40 %% Problem 3
41
42 for iny = 1:size(ny_list,2)
43     ny = ny_list(iny);
```

```

44     order = 2;
45     [coords, edges, triangle] = mesh(ny,order,false);
46
47     Ex = coords(1,:);
48     Ey = coords(2,:);
49
50     nnodes_u = size(coords,2);
51     vertices = [triangle(1,:),triangle(3,:),triangle(5,:)];
52     unique_vertices = unique(vertices);
53     nnodes_p = size(unique_vertices,2);
54
55     ndofs = nnodes_u*2+nnodes_p;
56     nelelem = size(triangle,2);
57
58     bound_id = 4;
59
60     %% Case I
61
62     for inu=1:size(nu0,2)
63
64         nu = nu0(inu);
65         K = sparse(ndofs,ndofs);
66         F = zeros(ndofs,1);
67
68         for ielem = 1:nelelem
69             unodes = triangle(:,ielem);
70             pnodes = triangle(1:2:end,ielem);
71
72             ex = coords(1,unodes);
73             ey = coords(2,unodes);
74             [Ke]= Taylor_hood(ex,ey,E,nu,t);
75
76             edof = zeros(1,15);
77
78             for k = 1:6
79                 n = unodes(k);
80                 edof(2*k-1:2*k) = [2*n-1,2*n];
81             end
82
83             for k= 1:3
84                 n=pnodes(k);
85                 np= find(unique_vertices==n)+nnodes_u*2;
86                 edof(2*size(triangle,1)+k)= np;
87             end
88             Edof(ielem,:) = edof;
89             K(edof,edof)=K(edof,edof)+Ke;
90         end
91
92         [funit,gamma_L] = Loadvector_interpolation_Taylor_hood(coords,
93             edges,bound_id,ndofs);
94
95         F_T = F + (-Tval)*funit;
96
97         fixed_dofs = [];
98
99         left_edge = find(edges(end,:)==5);
100        for le = 1:size(left_edge,2)

```

```

101         n1 = edges(1,left_edge(1,le));
102         n2 = edges(2,left_edge(1,le));
103
104         fixed_dofs = [fixed_dofs,2*n1-1,2*n2-1];
105     end
106
107     bottom_edge = find(edges(end,:)==1);
108
109     for be = 1:size(bottom_edge,2)
110         n1 = edges(1,bottom_edge(1,be));
111         n2 = edges(2,bottom_edge(1,be));
112
113         fixed_dofs = [fixed_dofs,2*n1,2*n2];
114     end
115
116
117     fdof = unique(fixed_dofs);
118     free_dof = setdiff(1:ndofs,fdof);
119
120     u= zeros(ndofs,1);
121
122     K_free = K(free_dof,free_dof);
123     F_free = F_T(free_dof);
124
125     u(free_dof) = K_free\F_free;
126
127     ueval(inu,iny) = -(1/(gamma_L))*(funit'*u);
128 end
129
130
131 %% Case II
132
133 for inu=1:size(nu0,2)
134
135     nu = nu0(inu);
136     K = sparse(ndofs,ndofs);
137     F = zeros(ndofs,1);
138
139     for ielem = 1:nelem
140         unodes = triangle(:,ielem);
141         pnodes = triangle(1:2:end,ielem);
142
143         ex = coords(1,unodes);
144         ey = coords(2,unodes);
145         [Ke]= Taylor_hood(ex,ey,E,nu,t);
146
147         edof = zeros(1,15);
148
149         for k = 1:6
150             n = unodes(k);
151             edof(2*k-1:2*k) = [2*n-1,2*n];
152         end
153
154         for k= 1:3
155             n=pnodes(k);
156             np= find(unique_vertices==n)+nnodes_u*2;
157             edof(2*size(triangle,1)+k)= np;
158         end

```

```

159         K(edof,edof)=K(edof,edof)+Ke;
160     end
161
162     [funit,gamma_L] = Loadvector_interpolation_Taylor_hood(coords,
163 edges,bound_id,ndofs);
164
165
166
167     fixed_dofs2 = [];
168
169     left_edge = find(edges(end,:)==5);
170
171     for le = 1:size(left_edge,2)
172         n1 = edges(1,left_edge(1,le));
173         n2 = edges(2,left_edge(1,le));
174
175         fixed_dofs2 = [fixed_dofs2,2*n1-1,2*n2-1];
176     end
177
178     bottom_edge = find(edges(end,:)==1);
179
180     for be = 1:size(bottom_edge,2)
181         n1 = edges(1,bottom_edge(1,be));
182         n2 = edges(2,bottom_edge(1,be));
183
184         fixed_dofs2 = [fixed_dofs2,2*n1,2*n2];
185     end
186
187     top_left_edge = find(edges(end,:)==4);
188     pres_dofs = [];
189     for tle = 1:size(top_left_edge,2)
190         n1 = edges(1,top_left_edge(1,tle));
191         n2 = edges(2,top_left_edge(1,tle));
192
193         fixed_dofs2 = [fixed_dofs2,2*n1,2*n2];
194         pres_dofs = [pres_dofs, 2*n1, 2*n2];
195     end
196
197     fixdof2 = unique(fixed_dofs2);
198     pdof = unique(pres_dofs);
199     free_dof2 = setdiff(1:ndofs,fixdof2);
200
201     u2= zeros(ndofs,1);
202
203     u2(pdof)= -Uval;
204     K_f = K(free_dof2,free_dof2);
205     F_f = F(free_dof2,1);
206     K_fc = K(free_dof2,fixdof2);
207
208     u_f = K_f\(F_f-K_fc*u2(fixdof2));
209
210     u2(free_dof2)=u_f;
211
212     RF = K*u2;
213
214     T = 0;
215

```

```

216     for i = 1: size(pdof,2)
217         T = T+ RF(pdof(i));
218     end
219
220     Teval(inu,iny) = -T/gamma_L ;
221 end
222
223
224 end
225
226 figure; hold on; grid on;
227 colors = lines(numel(nu0));
228
229 for j = 1:numel(nu0)
230     plot(ny_list, ueval(j,:), '-o', ...
231         'Color', colors(j,:), 'LineWidth', 1.5, 'DisplayName', sprintf('
232         \\nu = %.3f', nu0(j)));
233 end
234
235 xlabel('Number of Elements in Y-direction');
236 ylabel('Average Displacement (m)');
237 title('Mesh Convergence for Uniform Traction Load Case');
238 legend(Location='best');
239
240 figure; hold on; grid on;
241 colors = lines(numel(nu0));
242
243 for j = 1:numel(nu0)
244     plot(ny_list, Teval(j,:), '-o', ...
245         'Color', colors(j,:), 'LineWidth', 1.5, 'DisplayName', sprintf('
246         \\nu = %.3f', nu0(j)));
247 end
248
249 xlabel('Number of Elements in Y-direction');
250 ylabel('Average Traction (Pa)');
251 title('Mesh Convergence for Pres. Displacement Load Case');
252 legend(Location='best');
253
254 %% Pressure Distribution
255
256 ny = 15;
257 order = 2;
258 [coords, edges, triangle] = mesh(ny,order,false);
259
260 Ex = coords(1,:);
261 Ey = coords(2,:);
262
263 nnodes_u = size(coords,2);
264 vertices = [triangle(1,:),triangle(3,:),triangle(5,:)];
265 unique_vertices = unique(vertices);
266 nnodes_p = size(unique_vertices,2);
267
268 ndofs = nnodes_u*2+nnodes_p;
269 nelem = size(triangle,2);
270
271 bound_id = 4;

```



```

272 %% Case I
273
274 for inu=1:size(nu0,2)
275
276     nu = nu0(inu);
277     K = sparse(ndofs,ndofs);
278     F = zeros(ndofs,1);
279
280     for ielem = 1:nelem
281         unodes = triangle(:,ielem);
282         pnodes = triangle(1:2:end,ielem);
283
284         ex = coords(1,unodes);
285         ey = coords(2,unodes);
286         [Ke]= Taylor_hood(ex,ey,E,nu,t);
287
288         edof = zeros(1,15);
289
290         for k = 1:6
291             n = unodes(k);
292             edof(2*k-1:2*k) = [2*n-1,2*n];
293         end
294
295         for k= 1:3
296             n=pnodes(k);
297             np= find(unique_vertices==n)+nnodes_u*2;
298             edof(2*size(triangle,1)+k)= np;
299         end
300
301         K(edof,edof)=K(edof,edof)+Ke;
302     end
303
304     [funit,gamma_L] = Loadvector_interpolation_Taylor_hood(coords,edges,
305     bound_id,ndofs);
306
307     F_T = F + (-Tval)*funit;
308
309     fixed_dofs = [];
310
311     left_edge = find(edges(end,:)==5);
312
313     for le = 1:size(left_edge,2)
314         n1 = edges(1,left_edge(1,le));
315         n2 = edges(2,left_edge(1,le));
316
317         fixed_dofs = [fixed_dofs,2*n1-1,2*n2-1];
318     end
319
320     bottom_edge = find(edges(end,:)==1);
321
322     for be = 1:size(bottom_edge,2)
323         n1 = edges(1,bottom_edge(1,be));
324         n2 = edges(2,bottom_edge(1,be));
325
326         fixed_dofs = [fixed_dofs,2*n1,2*n2];
327     end
328

```

```

329     fdof = unique(fixed_dofs);
330     free_dof = setdiff(1:ndofs,fdof);
331
332     u= zeros(ndofs,1);
333
334     K_free = K(free_dof,free_dof);
335     F_free = F_T(free_dof);
336
337     u(free_dof) = K_free\F_free;
338
339     ueval_c(inu,1) = -(1/(gamma_L))*(funit'*u);
340
341     p1 = zeros(nelem,1);
342
343     for ip = 1:nelem
344
345         pnodes = triangle(1:2:end,ip);
346         press_on_nodes_1 = zeros(numel(pnodes),1);
347         for k = 1:numel(press_on_nodes_1)
348             idx= find(unique_vertices==pnodes(k));
349             press_on_nodes_1(k) = u(nnodes_u*2+idx);
350         end
351         p1(ip,1)= mean(press_on_nodes_1);
352     end
353
354     figure; hold on; axis equal; view(2);
355     patch('Faces', triangle(1:2:end,:), 'Vertices', coords', ...
356         'FaceColor', 'flat', 'CData', p1, 'EdgeColor', 'none');
357     colormap(jet); colorbar;
358     title(['Pressure for Uniform Traction Load at \nu=',num2str(nu)]);
359
360 end
361
362 figure
363 semilogy(mu,U_1,LineWidth=1.5,DisplayName='\epsilon_{11}=0')
364 hold on
365 semilogy(mu,U_2,LineWidth=1.5,DisplayName='\sigma_{11}=0')
366 semilogy(nu0,ueval_c, LineWidth=1.5,DisplayName='Uniform Traction load')
367 hold off
368
369 xlabel ('Poisson Ratio')
370 ylabel ('Displacement (m)')
371 title('Average Displacement For CASE I')
372 legend (Location="best")
373
374 figure
375
376 semilogy(nu0,ueval_c, LineWidth=1.5,DisplayName='Uniform Traction load')
377 set(gca, 'FontSize');
378 xlabel ('Poisson Ratio')
379 ylabel ('Displacement (m)')
380 title('Average Displacement For CASE I')
381 legend (Location="best")
382
383 %% Case II
384
385 for inu=1:size(nu0,2)
386

```

```

387 nu = nu0(inu);
388 K = sparse(ndofs,ndofs);
389 F = zeros(ndofs,1);
390
391 for ielem = 1:nelem
392     unodes = triangle(:,ielem);
393     pnodes = triangle(1:2:end,ielem);
394
395     ex = coords(1,unodes);
396     ey = coords(2,unodes);
397     [Ke]= Taylor_hood(ex,ey,E,nu,t);
398
399     edof = zeros(1,15);
400
401     for k = 1:6
402         n = unodes(k);
403         edof(2*k-1:2*k) = [2*n-1,2*n];
404     end
405
406     for k= 1:3
407         n=pnodes(k);
408         np= find(unique_vertices==n)+nnodes_u*2;
409         edof(2*size(triangle,1)+k)= np;
410     end
411
412     K(edof,edof)=K(edof,edof)+Ke;
413 end
414
415 [funit,gamma_L] = Loadvector_interpolation_Taylor_hood(coords,edges,
416 bound_id,ndofs);
417
418 fixed_dofs2 = [];
419
420 left_edge = find(edges(end,:)==5);
421
422 for le = 1:size(left_edge,2)
423     n1 = edges(1,left_edge(1,le));
424     n2 = edges(2,left_edge(1,le));
425
426     fixed_dofs2 = [fixed_dofs2,2*n1-1,2*n2-1];
427 end
428
429 bottom_edge = find(edges(end,:)==1);
430
431 for be = 1:size(bottom_edge,2)
432     n1 = edges(1,bottom_edge(1,be));
433     n2 = edges(2,bottom_edge(1,be));
434
435     fixed_dofs2 = [fixed_dofs2,2*n1,2*n2];
436 end
437
438 top_left_edge = find(edges(end,:)==4);
439 pres_dofs = [];
440 for tle = 1:size(top_left_edge,2)
441     n1 = edges(1,top_left_edge(1,tle));
442     n2 = edges(2,top_left_edge(1,tle));
443

```

```

444     fixed_dofs2 = [fixed_dofs2, 2*n1, 2*n2];
445     pres_dofs = [pres_dofs, 2*n1, 2*n2];
446 end
447
448 fixdof2 = unique(fixed_dofs2);
449 pdof = unique(pres_dofs);
450 free_dof2 = setdiff(1:ndofs, fixdof2);
451
452 u2 = zeros(ndofs, 1);
453
454 u2(pdof) = -Uval;
455 K_f = K(free_dof2, free_dof2);
456 F_f = F(free_dof2, 1);
457 K_fc = K(free_dof2, fixdof2);
458
459 u_f = K_f \ (F_f - K_fc * u2(fixdof2));
460
461 u2(free_dof2) = u_f;
462
463 RF = K * u2;
464
465 T = 0;
466
467 for i = 1: size(pdof, 2)
468     T = T + RF(pdof(i));
469 end
470
471 Teval_c(inu, 1) = -T / gamma_L ;
472
473 p2 = zeros(nelem, 1);
474 % p2 = zeros(nnodes_u, 1);
475 % p2(unique_vertices) = u2(nnodes_u * 2 + 1:end);
476
477 for ip = 1:nelem
478
479     pnodes = triangle(1:2:end, ip);
480     press_on_nodes_2 = zeros(numel(pnodes), 1);
481     for k = 1: numel(press_on_nodes_2)
482         idx = find(unique_vertices == pnodes(k));
483         press_on_nodes_2(k) = u2(nnodes_u * 2 + idx);
484     end
485     p2(ip, 1) = mean(press_on_nodes_2);
486 end
487
488 figure; hold on; axis equal; view(2);
489 patch('Faces', triangle(1:2:end, :)', 'Vertices', coords', ...
490     'FaceColor', 'flat', 'CData', p2, 'EdgeColor', 'none');
491 colormap(jet); colorbar;
492 title(['Pressure for Prescribed Displacement Load at \nu=', num2str(
493     nu)]);
494 end
495
496 figure
497 semilogy(mu, T_1, LineWidth=1.5, DisplayName='\epsilon_{11}=0')
498 hold on
499 semilogy(mu, T_2, LineWidth=1.5, DisplayName='\sigma_{11}=0')
500 semilogy(nu0, Teval_c, LineWidth=1.5, DisplayName='Prescribed Displacement

```

```
        load')
501 hold off
502
503 xlabel ('Poisson Ratio')
504 ylabel ('Traction (Pa)')
505 title('Average Traction For CASE II')
506 legend (Location="best")
507
508 figure
509
510 semilogy(nu0, Teval_c, LineWidth=1.5, DisplayName='Prescribed Displacement
        Load')
511 set(gca, 'FontSize');
512 xlabel ('Poisson Ratio')
513 ylabel ('Traction (Pa)')
514 title('Average Traction For CASE II')
515 legend (Location="best")
```

B.2 Load Interpolation for Quadratic Elements

```

1 function [funit,gamma_L] = Loadvector_interpolation_Taylor_hood(coords,
   edges, bound_id, ndofs)
2
3 funit = zeros(ndofs,1);
4 gamma_L=0;
5
6 for ec = 1:size(edges,2)
7
8     id = edges(end,ec);
9
10    if id==bound_id
11        n1 = edges(1,ec);
12        n2 = edges(2,ec);
13        n3 = edges(3,ec);
14
15        x1 = coords(:,n1);
16        x2 = coords(:,n3);
17        L = norm(x2-x1);
18
19        gamma_L = gamma_L+L;
20
21        funit(2*n1)=funit(2*n1)+L/6;
22        funit(2*n2)=funit(2*n2)+2*L/3;
23        funit(2*n3)=funit(2*n3)+L/6;
24    end
25
26 end
27
28 end

```

B.3 Taylor Hood Element Routine

B.3.1 Stiffness Matrix

```

1 function [Ke] = Taylor_hood(ex,ey,E,nu,t)
2
3 ngp = 3; % Number of Gauss Points
4
5 G = E/(2*(1+nu));
6 K = E/(3*(1-2*nu));
7
8 nodes = size(ex,2);
9 ndofs_u = nodes*2;
10 ndofs_p = nodes/2;
11
12 Xe= zeros(2,6);
13 for i = 1:6
14     Xe(1,i)=ex(i);
15     Xe(2,i)=ey(i);
16 end
17
18 xi_v = [(1/6) (1/6) (2/3);
19         (1/6) (2/3) (1/6)];
20
21 Se = zeros(ndofs_u,ndofs_u);
22 Ce = zeros(ndofs_u,ndofs_p);
23 Be = zeros(ndofs_p,ndofs_p);
24
25 A = diag([2 2 1]);
26 H_v = ones(1,ngp)/6;
27
28 for j=1:ngp
29     xi = xi_v(:,j);
30     H = H_v(j);
31     [B_eps,B_div,B_dev,N_p,detFisop]= Be_Taylor_Hood_implementation(xi,
32     Xe(:,1),Xe(:,2),Xe(:,3),Xe(:,4),Xe(:,5),Xe(:,6));
33     Se = Se + B_eps'*G*A*B_dev*detFisop*H*t;
34     Ce = Ce + B_div'*N_p*detFisop*H*t;
35     Be = Be + N_p'*(1/K)*N_p*detFisop*H*t;
36 end
37 Ke = [Se, -Ce;
38     -Ce', -Be];

```

B.3.2 Supporting Code

```

1 clear all
2 close all
3 clc
4
5 xi=sym('xi',[2,1],'real');
6 N1_u=(1-xi(1)-xi(2))*(1-2*xi(1)-2*xi(2));
7 N3_u=xi(1)*(2*xi(1)-1);
8 N5_u=xi(2)*(2*xi(2)-1);
9 N2_u=4*xi(1)*(1-xi(1)-xi(2));
10 N4_u=4*xi(1)*xi(2);
11 N6_u=4*xi(2)*(1-xi(1)-xi(2));

```

```

12
13 dN1_dxi=gradient(N1_u,xi);
14 dN2_dxi=gradient(N2_u,xi);
15 dN3_dxi=gradient(N3_u,xi);
16 dN4_dxi=gradient(N4_u,xi);
17 dN5_dxi= gradient(N5_u,xi);
18 dN6_dxi = gradient(N6_u,xi);
19
20 N1_p = 1-xi(1)-xi(2);
21 N2_p = xi(1);
22 N3_p = xi(2);
23
24 Xe1=sym('xe1',[2,1],'real');
25 Xe2=sym('xe2',[2,1],'real');
26 Xe3=sym('xe3',[2,1],'real');
27 Xe4=sym('xe4',[2,1],'real');
28 Xe5=sym('xe5',[2,1],'real');
29 Xe6=sym('xe6',[2,1],'real');
30
31 X=N1_u*Xe1+N2_u*Xe2+N3_u*Xe3+N4_u*Xe4+N5_u*Xe5+N6_u*Xe6;
32
33 Fisop=jacobian(X,xi);
34 detFisop=det(Fisop);
35
36 dN1_dX=simplify(inv(Fisop)*dN1_dxi);
37 dN2_dX=simplify(inv(Fisop)*dN2_dxi);
38 dN3_dX=simplify(inv(Fisop)*dN3_dxi);
39 dN4_dX=simplify(inv(Fisop)*dN4_dxi);
40 dN5_dX=simplify(inv(Fisop)*dN5_dxi);
41 dN6_dX=simplify(inv(Fisop)*dN6_dxi);
42
43 B_eps = [dN1_dX(1) 0 dN2_dX(1) 0 dN3_dX(1) 0 dN4_dX(1) 0 dN5_dX(1) 0
           dN6_dX(1) 0;
           0 dN1_dX(2) 0 dN2_dX(2) 0 dN3_dX(2) 0 dN4_dX(2) 0 dN5_dX(2) 0
           dN6_dX(2);
           dN1_dX(2) dN1_dX(1) dN2_dX(2) dN2_dX(1) dN3_dX(2) dN3_dX(1) dN4_dX(
           2) dN4_dX(1) dN5_dX(2) dN5_dX(1) dN6_dX(2) dN6_dX(1)];
46
47 B_div = [dN1_dX(1) dN1_dX(2) dN2_dX(1) dN2_dX(2) dN3_dX(1) dN3_dX(2)
           dN4_dX(1) dN4_dX(2) dN5_dX(1) dN5_dX(2) dN6_dX(1) dN6_dX(2)];
48
49 B_dev = B_eps - 1/3*[1;1;0]*B_div;
50
51 N_p = [N1_p N2_p N3_p];
52
53 matlabFunction(B_eps,B_div,B_dev,N_p,detFisop,'File','
    Be_Taylor_Hood_implementation','Vars',{xi,Xe1,Xe2,Xe3,Xe4,Xe5,Xe6})

```